NAME : Himanshu Dubey
REG. NO. : 24BSA10376

SOFTWARE : LINUX KERNEL

REPORT: THE OPEN SOURCE AUDIT, LINUX KERNEL

1. Introduction

The modern digital world relies on layers of software that often go unnoticed by users. At the heart of most computing systems is a key component: the operating system kernel. The Linux Kernel is one of the most significant open-source projects ever created, powering servers, smartphones, supercomputers, and embedded systems.

Unlike proprietary systems, the Linux Kernel was developed openly and collaboratively. This project seeks to explore not just its technical role but also its philosophical foundations, licensing model, and its effect on the global software ecosystem.

PART A — ORIGIN AND PHILOSOPHY

A1. Origin Story

The Linux Kernel was initiated in 1991 by Linus Torvalds, a student at the University of Helsinki. At that time, powerful operating systems like UNIX were either costly or had restrictions. Another option, MINIX, was available for educational use but had limitations in licensing and functionality.

Torvalds wanted a system that was free to use, could operate on personal hardware, and allowed for modification and experimentation.

Instead of creating a closed system, he made his work public. This decision turned Linux into a collaborative project where developers around the world contributed improvements.

The main issue Linux addressed was accessibility—it provided a Unix-like system to anyone, regardless of financial or institutional barriers.

A2. License Analysis

The Linux Kernel is licensed under the GNU General Public License Version 2 (GPLv2).

The Four Freedoms are:

- Freedom to run the program for any purpose
- Freedom to study how it works
- Freedom to modify it
- Freedom to distribute copies

The GPL is a copyleft license. This means that if someone modifies Linux and shares it, they must also provide their source code. This prevents companies from taking the code and making it permanently closed.

Can companies sell it? Yes, but they must provide the source code and cannot limit user freedoms.

GPL vs. MIT:
- GPL is strict and enforces openness.
- MIT is permissive and allows for closed derivatives.

Free Beer vs. Free Speech:
Linux is “free as in freedom,” not just in price. You can control and modify it, not just use it without charge.

A3. Ethics of Open Source

Open source embodies a philosophy of shared knowledge and collaboration. However, it raises important ethical questions.

Should all software be open source?

For:
- Encourages transparency
- Improves security through public review
- Speeds up innovation

Against:
- Companies might lose their competitive edge
- Some projects may need funding models that conflict with openness

Corporate Use of Open Source

Companies like Google and Red Hat build products based on Linux.

This brings up a question: is it fair to profit from community work?

A balanced view:
- Positive: companies contribute back and support development.
- Negative: risk of exploiting unpaid contributors.

“Standing on the Shoulders of Giants”

Linux shows how innovation builds on existing work. Open source promotes progress by removing barriers rather than starting from scratch.

<u>PART B — LINUX FOOTPRINT</u>

The Linux Kernel operates at the core of the operating system and is always present in a running Linux system.

Installation

The kernel is typically installed using package managers such as:

- apt (Debian/Ubuntu)
- yum or dnf (Red Hat-based systems)

Key Directories:
- /boot contains the kernel image (vmlinuz)
- /lib/modules/ includes kernel modules
- /proc offers a runtime interface to the kernel

These directories allow interaction with and extension of kernel functionality.

User and Privileges

The kernel operates in kernel space, separate from user applications.

This separation prevents user programs from accessing hardware directly and enhances system stability and security.

Service Management

The kernel is not a service. It is loaded during system boot by a bootloader like GRUB.

Update Model

Kernel updates come from:

- Linux distributions
- Community contributions

Developers submit patches that are reviewed and merged into the main kernel line.

PART C — FOSS ECOSYSTEM

The Linux Kernel is part of a large ecosystem, not isolated.

Dependencies include:
- GNU tools
- Compilers like GCC
- Standard libraries

What It Enables

The kernel supports technologies such as:
- Docker (uses namespaces, cgroups)
- Android OS
- Cloud infrastructure

Community and Governance

The project is managed by the Linux Foundation. Thousands of developers worldwide contribute, making it one of the largest collaborative software projects.

PART D — OPEN SOURCE VS PROPRIETARY

Comparison with: Microsoft Windows

Dimension	Linux Kernel	Windows Kernel
Cost	Free	Paid
Transparency	Full Source Available	Closed
Security	Auditable by anyone	Limited Visibility
Flexibility	Highly Customized	Restricted
Support	Community+Enterprise	Corporate

Final Verdict

The Linux Kernel offers unmatched flexibility, transparency, and scalability. It is ideal for servers, development environments, and embedded systems. However, it requires technical knowledge and may not be as user-friendly as proprietary options.

In practice, Linux is strongly recommended for performance-critical and customizable systems. Contributing to such a project would be valuable for learning and being part of a global developer community.

PART E — SHELL SCRIPT IMPLEMENTATION

Script 1 — System Identity Report

Objective : The aim of this script is to show key system information in a structured format, serving as a system introduction screen.

Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Your Name | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="HIMANSHU"
SOFTWARE_CHOICE="Linux Kernel"

# --- System Info ---
KERNEL=$(uname -r)
KERNEL_FULL=$(uname -a)
```

```
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE=$(date)

# Get distro name
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d "")

# --- Display ---
echo "========================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================"
echo "Software       : $SOFTWARE_CHOICE"
echo "Distro         : $DISTRO"
echo "Kernel Version : $KERNEL"
echo "User           : $USER_NAME"
echo "Home Directory : $HOME_DIR"
echo "Uptime         : $UPTIME"
echo "Date & Time    : $DATE"
echo "License        : GNU General Public License v2 (GPLv2)"
echo "========================================"
```

Explanation :

This script uses basic shell scripting concepts like variables, command substitution, and formatted output.

- uname -r retrieves the kernel version.
- whoami identifies the current user.
- uptime -p shows system uptime.
- /etc/os-release provides details about the Linux distribution.

Variables are used to store and reuse system information.

The script also includes a reference to the Linux Kernel license (GPLv2), connecting technical output with open-source philosophy.

Concepts Used:
- Variables
- Command substitution ($())
- Echo formatting

Output:

```
=========================================
Software        : Linux Kernel
Distro          : Linux Mint 22.3
Kernel Version  : 6.17.0-19-generic
User            : cipher
Home Directory  : /home/cipher
Uptime          : up 1 hour, 4 minutes
Date & Time     : Monday 30 March 2026 08:46:42 PM IST
License         : GNU General Public License v2 (GPLv2)
=========================================
```

Script 2 — Linux Kernel Package Inspector

Objective: This script checks if the Linux Kernel package is installed and shows its version and details.

Code:

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="linux-image-$(uname -r)"

echo "Checking Linux Kernel package..."

# Check using dpkg (Debian/Ubuntu)
if dpkg -l | grep -q "$PACKAGE"; then
    echo "$PACKAGE is installed."
    dpkg -l | grep "$PACKAGE"
else
    echo "$PACKAGE not found via dpkg."
fi

# Additional kernel info
echo ""
echo "Kernel Details:"
uname -r
uname -v

# Case statement (philosophy notes)
case "linux" in
    linux)
        echo "Linux Kernel: The open foundation of modern computing."
        ;;
    *)
        echo "Unknown package"
        ;;
esac
```

Explanation :

The script uses package management tools to verify installation status.

- dpkg -l checks installed packages.
- grep filters relevant output.
- uname -r dynamically fetchs the current kernel version.

A case statement displays a short philosophical note about the Linux Kernel.

This script shows how to inspect system-level software using command-line tools.

Concepts Used:
- Conditional statements (if-then-else)
- Case statement
- Pipes (|) and filtering

Output :

Script 3 — Disk and Permission Auditor

```
Checking Linux Kernel package...
linux-image-6.17.0-19-generic is installed.
ii  linux-image-6.17.0-19-generic             6.17.0-19.19~24.04.2
          amd64        Signed kernel image generic

Kernel Details:
6.17.0-19-generic
#19~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC Fri Mar  6 23:08:46 UTC 2
Linux Kernel: The open foundation of modern computing.
```

Objective : This script examines important system directories and reports their disk usage, permissions, and ownership.

Code:

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/boot" "/lib/modules")

echo "Directory Audit Report"
echo "--------------------------------------"

for DIR in "${DIRS[@]}"; do
   if [ -d "$DIR" ]; then
      PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
      SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
      echo "$DIR => Permissions: $PERMS | Size: $SIZE"
   else
      echo "$DIR does not exist"
   fi
done
```

```
# Kernel-specific check
KERNEL_DIR="/lib/modules/$(uname -r)"

echo ""
echo "Kernel Module Directory Check:"
if [ -d "$KERNEL_DIR" ]; then
    ls -ld $KERNEL_DIR
else
    echo "Kernel module directory not found!"
fi
```

Explanation :

The script loops through a predefined list of directories and checks their properties.

A for loop iterates through directories, while ls -ld extracts permission and ownership information. du -sh calculates disk usage. awk and cut extract specific fields.

It also includes a kernel-specific check for /lib/modules, where kernel modules are located.

Concepts Used:
- Loops (for)
- Text processing (awk, cut)
- File system inspection

Output:

```
Directory Audit Report
-------------------------------------
/etc => Permissions: drwxr-xr-x root root | Size: 18M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 449M
/home => Permissions: drwxr-xr-x root root | Size: 6.4G
/usr/bin => Permissions: drwxr-xr-x root root | Size: 202M
/tmp => Permissions: drwxrwxrwt root root | Size: 260K
/boot => Permissions: drwxr-xr-x root root | Size: 327M
/lib/modules => Permissions: drwxr-xr-x root root | Size: 493M

Kernel Module Directory Check:
drwxr-xr-x 5 root root 4096 Mar 16 01:30 /lib/modules/6.17.0-19-generic
```

Script 4 — Kernel Log File Analyzer

Objective : This script analyzes a log file and counts occurrences of a specific keyword, aiding in system monitoring.

Code:

```
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"kernel"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Check if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "Log file is empty. Retrying in 2 seconds..."
    sleep 2
fi

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Explanation:

The script processes log files line by line to identify specific patterns.

It accepts an input file via a command-line argument ($1) and uses a while read loop to go through file contents. grep -i performs case-insensitive matching. A counter variable tracks occurrences.

Additionally, the script shows the last five matching log entries, providing quick insights into recent system events.

Concepts Used:
- While loop
- Conditional checks
- Command-line arguments

Output:

```
Keyword 'kernel' found 2154 times in /var/log/syslog

Last 5 matching lines:
grep: /var/log/syslog: binary file matches
2026-03-30T19:45:34.829695+05:30 cipher-Latitude-5480 kernel: wlp1s0: associated
2026-03-30T19:46:02.824385+05:30 cipher-Latitude-5480 kernel: warning: `ThreadPo
olForeg' uses wireless extensions which will stop working for Wi-Fi 7 hardware;
use nl80211
2026-03-30T20:42:21.273437+05:30 cipher-Latitude-5480 kernel: workqueue: delayed
_fput hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
2026-03-30T20:42:21.290330+05:30 cipher-Latitude-5480 kernel: workqueue: delayed
_fput hogged CPU for >10000us 5 times, consider switching to WQ_UNBOUND
2026-03-30T20:45:16.025338+05:30 cipher-Latitude-5480 kernel: workqueue: delayed
_fput hogged CPU for >10000us 7 times, consider switching to WQ_UNBOUND
```

Script 5 — Open Source Manifesto Generator

Objective : This script creates a personalized open-source philosophy statement based on user input.

Code :

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
USER=$(whoami)
OUTPUT="manifesto_$USER.txt"

# Create manifesto
echo "----- Open Source Manifesto -----" > $OUTPUT
echo "Date: $DATE" >> $OUTPUT
echo "" >> $OUTPUT

echo "I use $TOOL daily, and it reminds me that software should be built with $FREEDOM at its core." >> $OUTPUT
echo "I believe in sharing knowledge openly, and one day I will create $BUILD and make it accessible to everyone." >> $OUTPUT
echo "The Linux Kernel and the open-source community inspire me to contribute, learn, and grow together." >> $OUTPUT

echo ""
```

```
echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

Explanation : The script interacts with the user and generates a text file containing a summary.

The read command gathers user input. Variables hold responses. The date command adds a timestamp. Output is written to a file using > and >>.

This script connects technical scripting with open-source philosophy by prompting users to reflect on freedom and sharing.

Concepts Used:
- User input
- File handling
- String concatenation

Output :

```
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: Linux
2. In one word, what does 'freedom' mean to you? Self-Decisions
3. Name one thing you would build and share freely: A good project

Manifesto saved to manifesto_cipher.txt
----- Open Source Manifesto -----
Date: 30 March 2026

I use Linux daily, and it reminds me that software should be built with Self-Dec
isions at its core.
I believe in sharing knowledge openly, and one day I will create A good project
and make it accessible to everyone.
The Linux Kernel and the open-source community inspire me to contribute, learn,
and grow together.
```